Gap Lesson — Resilience, Scale and Safeguards

Monday 26 Oct 2026 · Closes skills missed inside studied tasks: **1.2.2–1.2.4, 1.4.1–1.4.3, 1.5.2, 1.5.5, 2.1.3–2.1.7, 2.2.2, 2.4.2–2.4.4** · Exam: Mon 9 Nov, 1:30 PM IST

Why this lesson exists
On 26 Oct we checked all 98 official skills against what had been taught. Several skills inside 'finished' tasks were missing. This lesson closes them: model routing, switching models without code changes, resilience (circuit breakers, graceful degradation), the custom-model lifecycle, LLM hosting, vector search at scale, query handling, agent safeguards, MCP hosting, streaming to browsers, API rate limiting and tracing.

Routing mini-lesson — Skill 2.4.4: intelligent model routing

Real applications receive mixed requests: some easy, some hard. Sending everything to a large model wastes money; sending everything to a small one gives poor answers on hard requests. **Model routing** chooses the model per request — the cheapest one that can handle it well.

```
                          +--> small, cheap model     (easy / short requests)
request --> ROUTER -----+--> code-specialised model (programming questions)
                          +--> large, capable model   (complex reasoning)
```

① Static routing in application code

Fixed rules decide the model from something known in advance (feature, endpoint, customer tier), usually a configuration map. Better practice: keep the map in **AWS AppConfig** or **Systems Manager Parameter Store** so a feature can move to another model without redeploying. Example: premium customers → large model, free tier → small model. Simple and predictable, but blind to request content.

```
MODEL_FOR_FEATURE = {"faq_answer": "amazon.nova-micro-v1:0", "email_summary": "amazon.nova-lite-v1:0",
                     "contract_review": "<large model ID>"}
model_id = MODEL_FOR_FEATURE[feature]   # with Converse, only modelId changes
```

② Content-based routing with AWS Step Functions

```
request -> [Classify: small model or Comprehend -> 'code' | 'legal' | 'general']
          -> [Choice state]
               'code'    -> Bedrock InvokeModel: code-specialised model
               'legal'   -> Bedrock InvokeModel: large reasoning model (+ legal KB)
               'general' -> Bedrock InvokeModel: small general model
          -> [Return / store answer]  (built-in retries, execution history = audit trail)
```

Exam cue: 'route by request content to specialised FMs, auditable' → Step Functions classify + Choice.

③ Routing based on metrics

- **Amazon Bedrock Intelligent Prompt Routing** (managed): call a **prompt router** instead of a model; per prompt it predicts which model in a family gives good-enough quality and sends it to the cheaper one when quality is about equal. Exam cue: 'automatically pick the most cost-effective model in a family per prompt, least effort'.
- **Your own metrics-based router** using CloudWatch latency/error/throttle/cost metrics: **fallback** on throttling or latency spikes; **weighted A/B** (90% current, 10% candidate); **cascading** (small model first, escalate to large if the answer fails a quality check).

④ API Gateway with request transformations

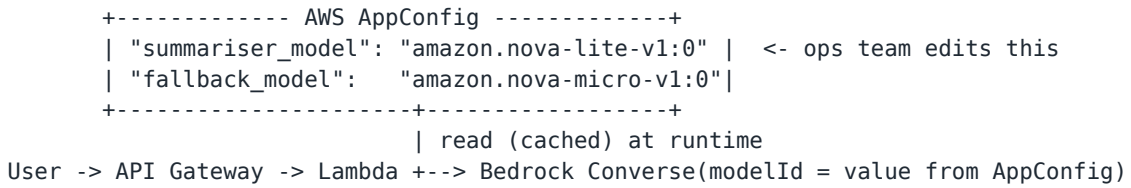
Route at the API layer: **path-based routes** (/v1/summarise vs /v1/analyse), and **mapping templates** that rewrite the request (insert the model ID, reshape the body) based on path, a header such as x-tier: premium, or a **stage variable**. Clients never need to know which model is used.

Scenario	Choose
Fixed model per feature or tier; simplest	Static routing (config map / AppConfig)
Route by request content, auditable	Step Functions: classify → Choice → model
Cheaper model in a family per prompt, least effort	Bedrock Intelligent Prompt Routing
Switch on latency/errors/throttling; A/B; escalate on low quality	Metrics-based routing (fallback, weighted, cascade)
Routing by path/header/stage, invisible to clients	API Gateway with request transformations

Part A — Resilience and the model lifecycle

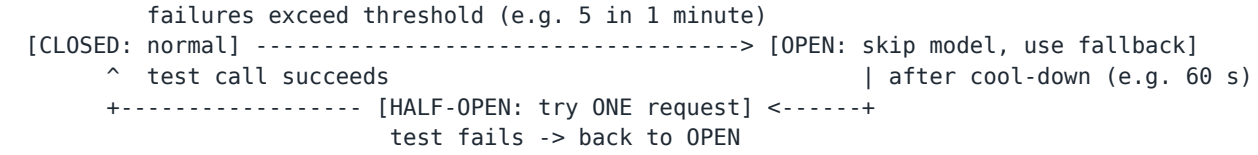
A1. Switching models without code changes (Skill 1.2.2)

Even a one-line code change means edit, test and redeploy. During an incident you need minutes. Keep the model choice in configuration: **AWS AppConfig** stores settings and **feature flags** your app reads at runtime (the AppConfig Lambda extension caches them). Changes can be **validated**, rolled out **gradually** (e.g. 10% first) and **rolled back automatically** when a CloudWatch alarm fires. API Gateway stage variables can switch at the API layer.



A2. Resilient AI systems (Skill 1.2.3)

① **Circuit breaker** — stop calling a failing service for a while and use an alternative; it **fails fast** instead of making every user wait for timeouts, and avoids piling load on the failing service.



On AWS: **Step Functions** checks circuit state stored in **DynamoDB**; OPEN → fallback branch; CLOSED → call the model and record failures.

- ② **Cross-region inference** also covers **models not available in your Region**: the profile routes to a Region where the model exists.
- ③ **Cross-Region deployment** of the whole app (Lambda, API Gateway, KB in two Regions) with **Route 53 failover**.
- ④ **Graceful degradation**: reduced but useful service instead of errors — smaller fallback model, cached answers, a clear 'AI answers delayed' message, answering without RAG plus a warning, skipping a failing tool.

A3. Custom model lifecycle (Skill 1.2.4)

LoRA (Low-Rank Adaptation) is a **parameter-efficient fine-tuning (PEFT)** technique: freeze the base model and train small **adapters**. Training is cheaper and faster, adapters are megabytes, and one base model on one SageMaker endpoint can serve many adapters.

```
FULL FINE-TUNING: [7B model - ALL weights changed] -> new ~14 GB copy per use case
LoRA:             [7B base model - FROZEN, shared] + [adapter A 20 MB] (legal)
                                                   + [adapter B 20 MB] (medical)
                                                   + [adapter C 20 MB] (support)
```

SageMaker Model Registry stores every model version with an approval status (Pending / Approved / Rejected). An automated pipeline (**SageMaker Pipelines** or **AWS CodePipeline**, triggered via EventBridge) deploys only Approved versions; rollback = redeploy the previous approved version.

```
Train v3 -> register (PendingManualApproval) -> reviewer/tests approve -> Approved
         -> EventBridge event -> deployment pipeline deploys v3
```

- **Deployment guardrails** on SageMaker endpoints: **blue/green** (old and new side by side), **canary** (small slice first), **linear** (equal steps), **auto-rollback** when a CloudWatch alarm fires.
- **Retiring models**: Bedrock models move **Active → Legacy → End-of-Life**; after EOL calls fail. Track dates, evaluate the replacement early, switch via configuration, canary, keep rollback until proven.

A4. What makes deploying LLMs different (Skill 2.2.2)

GPU memory ≈ parameters × bytes per parameter, plus the **KV cache** (working memory per conversation, growing with context length and concurrent users).

Precision	Bytes / parameter	13B model	70B model
32-bit	4	52 GB	280 GB
16-bit (usual)	2	**26 GB**	140 GB
8-bit	1	13 GB	70 GB
4-bit	0.5	6.5 GB	35 GB

- **Quantisation** (8/4-bit) shrinks memory 2–4× with a small quality loss; **tensor parallelism** splits a model across several GPUs.
- **Containers**: **Large Model Inference (LMI)** containers (vLLM, TensorRT-LLM) or Hugging Face **Text Generation Inference (TGI)** handle batching, streaming, quantisation, multi-GPU.
- **Model loading** from S3 can take minutes — hurts auto-scaling; keep warm instances, pre-cache artefacts, use smaller/quantised models.
- If Bedrock offers the model, Bedrock handles all of this — usually the 'least operational overhead' answer.

Part B — Vector stores at scale and query handling

B1. More vector architectures (Skill 1.4.1)

- **OpenSearch Service + neural plugin**: OpenSearch itself calls an embedding model (e.g. Bedrock via a connector) at ingest and query time; you send text. 'Topic-based segmentation' = topic-specific indexes.
- **RDS / Aurora PostgreSQL + pgvector with S3 documents**: documents in S3; chunks, embeddings and metadata in SQL — join vectors with business data.
- **DynamoDB alongside a vector DB**: DynamoDB holds metadata/records (fast ID lookup); OpenSearch does similarity search; zero-ETL sync. DynamoDB is still **not** a vector store.

```
WITHOUT neural plugin: code -> Bedrock (embed) -> vector -> code -> OpenSearch
WITH neural plugin:    code -> OpenSearch -> (OpenSearch calls Bedrock to embed) -> store/search
DynamoDB pairing:      query -> OpenSearch (vector search) -> top IDs -> DynamoDB (fetch records)
```

B2. Metadata frameworks (Skill 1.4.2)

```
s3://kb-docs/refund-policy.pdf
s3://kb-docs/refund-policy.pdf.metadata.json
{"metadataAttributes": {"department": "finance", "doc_type": "policy", "author": "j.smith",
                        "effective_date": 20261001, "region": "EU"}}
```

- Filter at retrieval (equals, greater-than, in-list, starts-with): department = finance AND effective_date ≥ 20260101.
- Outdated answers → effective_date / version attribute + filter, or remove superseded documents and re-sync.
- Return author/date with answers for transparency; domain tags keep searches in the right content type.

B3. Vector search at scale (Skill 1.4.3)

Shards are pieces of one index spread across nodes. A query goes to **all** shards at once (scatter), each returns its best matches, and results are merged (gather) — speed comes from **parallelism**, not early exit. **Replicas** are copies of shards: they **serve reads** (more throughput) and **survive node failure**.

```
query --> sent to ALL shards at once
          node 1: shard 1 -> its top 10 --+
          node 2: shard 2 -> its top 10 --+--> merged -> overall top 10
          node 3: shard 3 -> its top 10 --+
```

- **Multi-index per domain** (legal, HR, product): smaller, faster, domain-specific settings/embeddings.
- **Hierarchical indexing**: search document summaries first, then chunks within the top documents.
- ANN algorithms (e.g. **HNSW**) trade recall vs latency (e.g. ef_search); **vector quantisation** cuts memory.

B4. Embedding dimensions and batching (Skill 1.5.2)

More dimensions (1024)	Fewer dimensions (256)
More nuance, usually slightly better retrieval	4× less storage, faster search, cheaper
Bigger index, more memory, higher cost	Slightly less precise

Decide by evaluating retrieval quality on your queries; consider domain fit and modality (Titan Multimodal Embeddings for images). Load millions of documents with **batch embedding generation** (parallel Lambda from S3/SQS, or batch inference). Queries and documents must use the same embedding model.

B5. Query handling (Skill 1.5.5)

```
Expansion:     "laptop won't charge" -> + "battery not charging", "power adapter fault" -> search all -> me
HyDE:          FM writes a hypothetical answer -> search with it
Decomposition: "Compare refund and shipping of Basic vs Premium" -> 4 sub-queries -> combine
Rewriting:     "And for premium?" -> "What is the refund window for the Premium plan?"
```

Step Functions pipeline: [Rewrite (FM)] → [Decompose / expand] → [**Map state**: retrieve each sub-query in parallel] → [Merge + rerank] → [Generate].

B6. How Bedrock offers query handling

Technique	Built into Knowledge Bases?	How
Query decomposition	Yes	RetrieveAndGenerate query transformation type QUERY_DECOMPOSITION
Follow-up questions	Yes	Pass the sessionId from the previous RetrieveAndGenerate call
Reranking	Yes	Reranking configuration with a Bedrock rerank model
Hybrid vs semantic	Yes	Search type setting (overrideSearchType)
Metadata filter — explicit	Yes	Pass a filter
Metadata filter — implicit	Yes	A model builds the filter from the question using attribute descriptions
Query expansion / HyDE	No single switch	Bedrock model call to rephrase, then Retrieve each and merge
Custom multi-step pipelines	Build it	Step Functions around the Retrieve API (Map state)

```
bedrock_agent_runtime.retrieve_and_generate(
    input={"text": "Compare refund and shipping for Basic vs Premium"},
    sessionId=previous_session_id,                                  # follow-ups keep context
    retrieveAndGenerateConfiguration={"type": "KNOWLEDGE_BASE", "knowledgeBaseConfiguration": {
        "knowledgeBaseId": "KB123", "modelArn": "<model ARN>",
        "orchestrationConfiguration": {"queryTransformationConfiguration": {"type": "QUERY_DECOMPOSITION"}},
        "retrievalConfiguration": {"vectorSearchConfiguration": {
            "numberOfResults": 10, "overrideSearchType": "HYBRID",
            "filter": {"equals": {"key": "department", "value": "finance"}}}}}})
```

Part C — Agent safeguards, MCP hosting, streaming, rate limiting, tracing

C1. Safeguarded agent workflows (Skill 2.1.3)

```
[Call model] -> [Choice: tool?] -> [Run tool] -> [Increment counter] -> [Choice: counter < 10 AND budget le
     ^------------------------------------ yes ------------------------------+
                                           no -> [Stop: best answer + 'limit reached' / escalate]
```

- **Stopping conditions**: max iterations, token/cost budget (+ CloudWatch billing alarm), total time, 'same tool failed 3 times'.
- **Timeouts**: per-tool Lambda timeout; Step Functions TimeoutSeconds on states and the workflow.
- **IAM boundaries**: least privilege per tool role; **permission boundaries** cap the maximum a role can get — limits damage if a prompt injection manipulates the agent.
- **Circuit breakers** for failing tools.

C2. Coordinating multiple models (Skill 2.1.4)

```
                +--> Model A --> answer A --+
question -------+--> Model B --> answer B --+--> AGGREGATION --> final answer
                +--> Model C --> answer C --+
```

Aggregation: **majority vote** (classification), **judge model** (LLM-as-a-judge picks best), **merge** complementary parts, **confidence-based** (disagreement → human review). Worth it for high-stakes accuracy; costs ~3× latency and money. A 'model selection framework' = routing logic.

C3. Humans in the loop (Skill 2.1.5)

```
User clicks thumbs up/down (+ comment) -> API Gateway POST /feedback -> Lambda -> DynamoDB
  store request ID, prompt, answer, model version, rating -> dashboards, bad-answer review, eval datasets
```

Human augmentation: send low-confidence or disputed outputs to people; **Amazon Augmented AI (A2I)** manages human review workflows. Approvals: Step Functions waitForTaskToken (Day 3).

C4. Reliable tool integrations (Skill 2.1.6)

```
{"name": "refund_order",
 "description": "Refunds an order. Only for orders in DELIVERED status, within 30 days.",
 "inputSchema": {"json": {"type": "object",
   "properties": {"order_id": {"type": "string", "pattern": "^ORD-[0-9]{6}$"},
                  "reason":   {"type": "string", "enum": ["damaged", "late", "wrong_item"]},
                  "amount":   {"type": "number", "minimum": 0, "maximum": 5000}},
   "required": ["order_id", "reason"]}}}
```

- Strict schemas (types, required, enum, pattern, min/max) prevent bad calls.
- **Validate again in Lambda** — never trust model-generated parameters; return clear toolResult errors.
- Retries with backoff for temporary failures; **idempotent** actions (unique request ID so a retry doesn't refund twice).
- Strands lets you customise behaviour around tool calls (logging, validation, approvals).

C5. Hosting MCP servers (Skill 2.1.7)

	AWS Lambda	Amazon ECS (often Fargate)
Best for	Stateless, lightweight, quick tools	Complex, stateful, long-running tools; heavy dependencies
Examples	Weather, order status, currency	Persistent DB connection pools, browser automation, long streams
Scaling / cost	Scales to zero; pay per call	Runs continuously; you size containers
Limits	15-minute max; no state between calls	No time limit; keeps state in memory

Use **MCP client libraries** (built into Strands) so every tool is called the same way; AgentCore Gateway can front Lambda functions as MCP tools.

C6. Real-time streaming to the browser (Skill 2.4.2)

```
Browser <=== WebSocket (stays open) ===> API Gateway WebSocket API -> Lambda -> Bedrock ConverseStream
        <-- token <-- token <-- token <--- Lambda posts each chunk to the connection
```

Option	What it is	Use when
WebSockets (API Gateway WebSocket API)	Persistent two-way connection	Interactive chat, interruptions, two-way
Server-sent events / Lambda response streaming	One-way stream over HTTP	Just stream the answer to the user
API Gateway chunked transfer encoding	HTTP response sent in pieces	REST-style APIs streaming tokens

C7. Rate limiting and tracing (Skill 2.4.3)

```
Abusive client --> API Gateway: usage-plan limit reached -> 429   (Bedrock quota protected)
Normal users   --> API Gateway --> Lambda --> Bedrock  OK

X-Ray trace for request abc123 (total 12.1 s)
  API Gateway 0.05 s | Lambda 12.0 s -> KB Retrieve 0.4 s, Bedrock call 11.4 s  <- bottleneck
```

- **API Gateway throttling**: rate and burst limits per API/stage/method; **usage plans with API keys** give per-client quotas; excess gets HTTP 429 before reaching Bedrock.

- **AWS X-Ray**: traces each request across services with timings and a service map — find the slow step.

Decision rules — gap lesson

1. Switch models without code change/redeploy → **AppConfig** (+ Lambda/API Gateway), gradual rollout, alarm rollback.
2. Stop hammering a failing model → **circuit breaker** (Step Functions + DynamoDB).
3. Model not in your Region → **cross-region inference**; Region outage → **multi-Region + Route 53 failover**.
4. Reduced service instead of errors → **graceful degradation**.
5. Many cheap variants of one model → **LoRA adapters** on a shared base endpoint.
6. Govern custom models → **Model Registry** + pipeline; safe rollout → **blue/green, canary, linear + auto-rollback**.
7. Model nearing end-of-life → evaluate, switch via config, canary, retire.
8. Self-hosting a big LLM → GPU memory sizing, **quantisation**, **tensor parallelism**, **LMI/TGI** containers.
9. OpenSearch embeds automatically → **neural plugin**; vectors + SQL joins → **pgvector**; DynamoDB = metadata only.
10. Latest version / department / author → **.metadata.json + filters** (explicit or implicit).
11. Slow huge index → **shards** (parallel), **replicas** (throughput, availability), multi-index, hierarchical.
12. Cheaper vector search → fewer dimensions or vector quantisation; millions of docs → batch embedding.
13. Vague → expansion; multi-part → decomposition; follow-ups → rewriting / sessionId; least effort → KB built-ins.
14. Runaway agents → stopping conditions + budgets; slow tools → timeouts; manipulation → least-privilege IAM.
15. High-stakes accuracy → ensemble + aggregation; ratings → API Gateway → Lambda → DynamoDB; review → A2I.
16. Bad tool params → strict schema + Lambda validation + clear errors + idempotency.
17. Light MCP tools → Lambda; complex/stateful/long → ECS (Fargate).
18. Two-way chat → WebSockets; one-way stream → SSE / chunked transfer; backend → Bedrock streaming.
19. One client exhausting capacity → **API Gateway throttling + usage plans**; find slow service → **X-Ray**.
20. Model routing → static config, Step Functions content-based, Intelligent Prompt Routing, metrics, API Gateway.

Your part checks — graded

Part	Your answers	Feedback
A	Circuit breaker vs retries; LoRA for 15 variants; 13B GPU memory	✓ ✓ — circuit breakers also fail fast; LoRA saves storage and needs one shared endpoint. Memory: 13B × 2 bytes = **26 GB** + KV cache (worked through together).
B	Latest-version fix; shards vs replicas; follow-up questions	✓ ½ ✓ — shards don't stop early: all shards are searched in parallel and merged; replicas also add read throughput.
C	Runaway agent safeguards; MCP host for 30-min stateful tool; WebSockets vs SSE	✓ ✓ ✓ — add token/cost budgets with billing alarms; Lambda's 15-minute limit rules it out.

Mistakes and learnings

Point	Correction
Shards 'stop early when a match is found'	All shards are searched in parallel (scatter-gather); speed = parallelism.
GPU memory sizing unknown	Memory ≈ parameters × bytes per parameter (16-bit = 2 bytes) + KV cache.
DynamoDB in RAG designs	Valid only for metadata/records next to a vector DB — never as the vector store.

Glossary — new terms

Term	Meaning
Model routing	Choosing the model per request to balance cost and quality.
AWS AppConfig / feature flag	Service for runtime settings; a flag is an on/off or value read while the app runs.
Systems Manager Parameter Store	AWS store for configuration values and secrets.
Stage variable	Per-stage setting in API Gateway (e.g. dev vs prod).
Mapping template	API Gateway rule that transforms requests/responses.
Intelligent Prompt Routing	Bedrock feature routing each prompt to the cheaper adequate model in a family.
Cascading	Try a small model first; escalate to a larger one if quality checks fail.
Circuit breaker (closed/open/half-open)	Stops calls to a failing service, uses a fallback, tests recovery.
Graceful degradation	Reduced but useful service instead of errors.
Route 53 failover	DNS routing that sends users to a healthy Region.
LoRA / PEFT / adapter	Parameter-efficient fine-tuning: train small adapters on a frozen base model.
SageMaker Model Registry	Catalogue of model versions with approval status.
SageMaker Pipelines / CodePipeline	Automated ML / CI-CD pipelines.
Blue/green, canary, linear	Deployment strategies shifting traffic gradually; auto-rollback on alarms.
Active / Legacy / End-of-Life	Bedrock model lifecycle stages.
KV cache	GPU working memory per conversation; grows with context and users.
Quantisation	Storing weights/vectors with fewer bits to save memory.
Tensor parallelism	Splitting one model across several GPUs.
LMI / TGI containers	Large Model Inference and Text Generation Inference serving containers.
OpenSearch neural plugin	OpenSearch calls an embedding model itself at ingest and query time.
Zero-ETL integration	Managed sync between services without pipeline code (e.g. DynamoDB → OpenSearch).
Sidecar .metadata.json	File next to a KB document holding its metadata attributes.
Implicit filtering	A model builds metadata filters from the user's question.
Shard / replica	Piece of an index on a node / copy of a shard for reads and availability.
ANN / HNSW / ef_search	Approximate nearest-neighbour search; HNSW algorithm; recall-vs-latency setting.
Query expansion / HyDE	Search with rephrasings / with a hypothetical answer.
Query decomposition / rewriting	Split multi-part questions / make follow-ups standalone.
Map state	Step Functions state running a step for every item, in parallel.
Stopping condition	Limit that ends an agent loop (iterations, budget, time).
Permission boundary	IAM feature capping the maximum permissions a role can have.
Ensemble / aggregation	Several models answer; combine by vote, judge, merge.
Amazon A2I (Augmented AI)	Managed human review workflows for ML outputs.
Idempotent	Repeating a request has the same effect as doing it once.

Term	Meaning
WebSocket API	API Gateway persistent two-way connections.
Server-sent events (SSE)	One-way server-to-browser stream over HTTP.
Chunked transfer encoding	HTTP response sent in pieces.
Lambda response streaming	Lambda sends its response progressively (e.g. via function URLs).
Usage plan / API key	API Gateway per-client quotas and throttling.
AWS X-Ray / service map	Distributed tracing with per-step timings; diagram of services.

Next
Gap-lesson quiz (new questions), then Day 6 — Guardrails and Responsible AI — from Part 1.